

Proyecto de nivelación — Plataforma de conversión documental a bundles OKF

ISIS4426	Desarrollo de Soluciones Cloud
Tipo de trabajo	Proyecto de nivelación en parejas
Duración	3 semanas
Backend	Go, obligatorio
Frontend	Tecnología a elección de la pareja
Despliegue	Docker y Docker Compose
Uso de agentes de IA	Permitido de manera abierta
Sustentación	Video de máximo 20 minutos

1. Propósito

El presente proyecto de nivelación tiene como finalidad que la pareja consolide competencias básicas en administración de sistemas, desarrollo de software, lenguaje Go y contenedores Docker, y las integre en una solución operativa de extremo a extremo. El dominio seleccionado es la conversión de documentos a bundles de conocimiento, un problema que exige separar la recepción de la solicitud del trabajo de cómputo, coordinar procesos asíncronos y preservar el aislamiento de los resultados por usuario. El énfasis recae en el ejercicio de dichas competencias sobre un caso representativo y no en la exactitud del conversor.

2. La tarea central

El proyecto consiste en construir una plataforma web multiusuario que reciba documentos, los procese de forma asíncrona mediante workers y produzca como resultado un bundle de conocimiento compatible con Open Knowledge Format. El backend se implementa en Go, el sistema se despliega mediante Docker Compose y la conversión no se ejecuta en ningún caso dentro de la petición HTTP de carga.

El flujo esperado puede resumirse así. Un usuario carga un documento, la API responde de inmediato con un identificador de trabajo, un worker procesa el documento en segundo plano y, una vez concluido, el usuario descarga un bundle validado.

3. El bundle OKF

Un bundle es una carpeta autocontenida que representa un documento convertido a unidades de conocimiento en formato Markdown. Para efectos de este proyecto es suficiente la siguiente definición operativa.

- `index.md` en la raíz, con la navegación y los datos del bundle.
- `log.md` en la raíz, con la trazabilidad de la conversión.
- Uno o más documentos de concepto en Markdown, uno por unidad lógica detectada.
- De forma opcional, un directorio `assets/` con recursos referenciados.

Estructura mínima, obligatoria incluso para un documento breve sin divisiones.

```
bundle/  
├─ index.md  
├─ log.md  
└─ documento.md
```

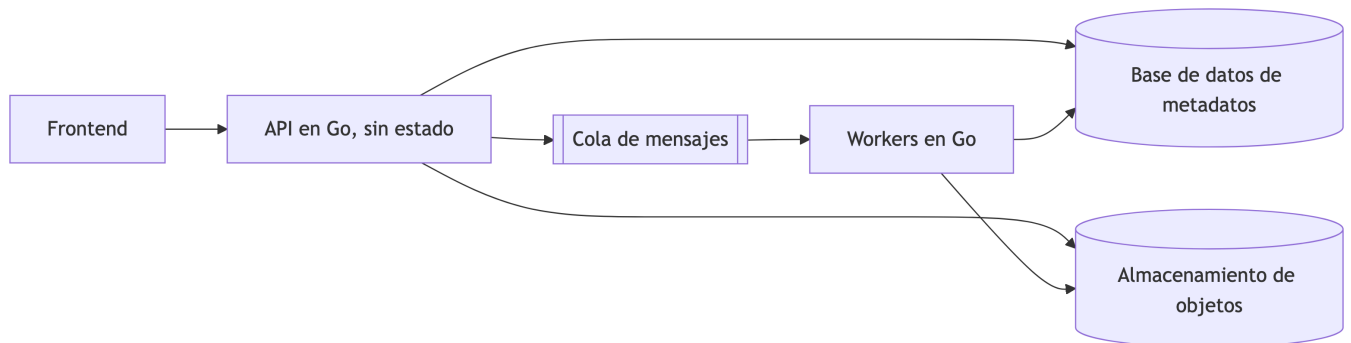
Estructura de un documento con varias unidades.

```
bundle/  
├─ index.md  
├─ log.md  
├─ capitulo-01.md  
├─ capitulo-02.md  
└─ capitulo-03.md
```

El archivo `index.md` enumera y enlaza los conceptos en el orden del documento de origen. El archivo `log.md` registra las operaciones efectuadas durante la conversión, las unidades detectadas, las transformaciones aplicadas y las validaciones superadas. Un resultado exitoso no corresponde nunca a un archivo Markdown aislado, sino al bundle completo.

4. Arquitectura requerida

La solución debe separar los servicios indicados a continuación y orquestarlos mediante Docker Compose.



Reglas de arquitectura obligatorias.

- La API no mantiene estado. No conserva trabajos ni archivos en memoria ni en el disco local del contenedor.
- La conversión se ejecuta en workers independientes, escalables sin necesidad de escalar la API.
- La entrega de trabajos se realiza mediante una cola de mensajes. La API publica el trabajo y retorna de inmediato.
- Los documentos originales y los bundles se almacenan fuera del disco efímero de los contenedores, en un almacenamiento de objetos o mecanismo equivalente.
- Los metadatos de usuarios, documentos, trabajos y bundles se persisten en una base de datos.

- El sistema completo se despliega con una sola invocación de `docker compose up`.

La elección concreta de la cola, el almacenamiento y la base de datos es libre. Cualquier alternativa disponible como imagen de Docker resulta admisible.

5. Alcance funcional

5.1 Alcance mínimo obligatorio

1. Registro y autenticación de usuarios. Un mecanismo de autenticación simple es suficiente, pues no constituye el eje del proyecto.
2. Carga de un documento que retorna de inmediato un identificador de trabajo, sin aguardar la conversión.
3. Encolamiento del trabajo y procesamiento por workers independientes de la API.
4. Segmentación del documento en unidades lógicas según su estructura.
5. Generación del bundle con `index.md`, `log.md` y al menos un documento de concepto.
6. Validación mínima del bundle antes de su publicación. Debe verificarse la estructura mínima y la resolución de los enlaces del índice.
7. Consulta del estado del trabajo y descarga del bundle completo en un paquete.
8. Aislamiento por propietario en toda operación. Ningún usuario accede a recursos de otro.
9. Persistencia de originales y bundles fuera del disco efímero de los contenedores.
10. Despliegue completo con un solo comando de Docker Compose.

En cuanto a los formatos de entrada, el sistema debe convertir al menos un formato con estructura detectable. Se sugieren Markdown, HTML o texto plano con encabezados. El soporte de formatos adicionales, como PDF, DOCX o EPUB, es opcional y se valora en la rúbrica.

5.2 Alcance opcional con valoración adicional

- Soporte de varios formatos de entrada.
- Extracción de imágenes u otros recursos a `assets/` con referencia desde los conceptos.
- Reintento idempotente de trabajos fallidos, con vínculo al trabajo anterior.
- Cancelación de trabajos en curso.
- Cálculo de conformidad OKF de forma separada de la validez de plataforma.
- Métricas y observabilidad básica del flujo de trabajos.
- Descarga por flujo para bundles de gran tamaño, sin materializar el paquete completo en memoria de la API.

6. Condiciones verificables

Las siguientes condiciones fundamentan la evaluación y deben demostrarse durante la sustentación.

- **Asincronía efectiva.** Al cargar un documento cuyo procesamiento es prolongado, la API responde con el identificador y el estado sin aguardar su conclusión. El cliente puede cerrar la conexión y el trabajo prosigue.
- **Documento breve.** Un documento corto sin divisiones produce `index.md`, `log.md` y un único concepto. El proceso no falla ni emite advertencias por el solo hecho de contener una única unidad.

- **Documento estructurado.** Un documento con varias secciones produce un concepto por unidad, enlazados en orden desde [index.md](#).
- **Bundle incompleto.** Ante la ausencia de [index.md](#) o [log.md](#), la validación falla, el bundle no se publica y no se habilita su descarga.
- **Aislamiento.** Si un usuario conoce el identificador de un recurso ajeno e intenta consultarlo o descargarlo, el servidor deniega la operación sin revelar información.
- **Ausencia de duplicados.** Si la cola entrega dos veces el mismo trabajo, se produce un único efecto final y, a lo sumo, un bundle publicado.

7. Restricciones técnicas

- El backend, tanto la API como los workers, se implementa íntegramente en Go.
- El frontend emplea la tecnología que la pareja determine.
- La totalidad de los componentes se ejecuta en contenedores y se orquesta mediante Docker Compose.
- El repositorio incluye un archivo [README](#) con las instrucciones de despliegue y prueba, y las variables de configuración se mantienen separadas del código.
- Se permite el uso abierto de agentes de inteligencia artificial para el diseño, la codificación, las pruebas y la documentación. Dicho uso debe declararse en la sustentación.

8. Entregables

1. Repositorio con la totalidad del código, el archivo [docker-compose.yml](#) y el [README](#).
2. Sistema que se despliega con un solo comando y queda operativo.
3. Video de sustentación con una duración máxima de 20 minutos.

La estructura obligatoria del video, alineada con la rúbrica, se detalla en la sección 10.

9. Rúbrica de evaluación

9. Rúbrica de evaluación

La calificación se distribuye en siete criterios, con un total de 100 puntos.

Criterio	Peso	Qué se evalúa
Empaquetado y despliegue	20%	Separación clara de servicios, API sin estado, orquestación completa con Docker Compose y despliegue con un solo comando. Persistencia fuera del disco efímero. Reproducibilidad a partir del README .
Procesamiento asíncrono y tolerancia a fallos	10%	La conversión no bloquea la petición HTTP. Empleo correcto de la cola, workers escalables de manera independiente, idempotencia frente a reentregas y una política de reintentos coherente.
Backend en Go y diseño de la API	20%	Calidad del código en Go, organización del proyecto, contratos de API claros y manejo de errores consistente.
Conversión documental y generación del bundle	15%	Segmentación razonable en unidades lógicas, preservación del orden y generación correcta de index.md , log.md y los conceptos. El soporte de más de un formato se valora dentro de este criterio.

Criterio	Peso	Qué se evalúa
Validación del bundle y clasificación del resultado	10%	Verificación de la estructura mínima y de los enlaces obligatorios antes de la publicación. Distinción entre resultado válido, válido con advertencias e inválido.
Aislamiento multi-usuario y control de acceso	10%	Autorización por propietario en todas las operaciones. Imposibilidad de acceder a recursos ajenos mediante la manipulación de identificadores.
Frontend funcional	15%	Permite ejecutar el flujo completo con claridad, incluida la carga, el seguimiento del estado y la descarga del bundle.

Escala por criterio. Cada criterio se califica en cuatro niveles, a saber, insuficiente, cumplimiento parcial, cumplimiento y desempeño esperado. Un criterio alcanza el nivel esperado cuando, además de operar correctamente, evidencia decisiones de diseño justificadas y robustez frente a los casos exigentes descritos en la sección 6.

10. Especificación del video de sustentación

El video constituye la evidencia principal de calificación. Debe organizarse de modo que cada criterio de la rúbrica pueda verificarse de forma directa. Un criterio que no se demuestre en el video no podrá acreditarse en su nivel superior, aun cuando el código lo contenga.

[!warning] Condición de entrega No presentar el video de sustentación equivale a no realizar la entrega del proyecto.

10.1 Requisitos formales

- Duración máxima de 20 minutos.
- La pantalla debe ser legible. El código y la interfaz deben visualizarse con claridad.
- El audio debe ser claro y ambos integrantes deben intervenir.
- La demostración se realiza sobre el sistema desplegado con Docker Compose. No se admiten capturas estáticas ni simulaciones.

10.2 Estructura recomendada

Cada segmento aporta evidencia a uno o más criterios de la rúbrica. Los tiempos constituyen una guía para no exceder los 20 minutos.

Segmento	Tiempo	Criterio de rúbrica	Qué debe verse en pantalla
1. Presentación e índice	1 min	Encuadre general	Integrantes, declaración del uso de agentes de inteligencia artificial e índice con las marcas de tiempo del video.
2. Arquitectura y decisiones	3 min	Arquitectura de nube y despliegue, Backend en Go	Diagrama de servicios, justificación de la cola, el almacenamiento y la base de datos, y explicación del carácter sin estado de la API.

Segmento		Tiempo	Criterio de rúbrica	Qué debe verse en pantalla
3. Despliegue re-producible		1,5 min	Arquitectura de nube y despliegue	Ejecución de un solo comando de Docker Compose desde un entorno limpio hasta dejar el sistema en operación.
4. Flujo completo		4 min	Conversión documental, Frontend funcional	Carga desde el frontend, retorno del identificador, seguimiento del estado y descarga del bundle.
5. Evidencia de asincronía		2 min	Procesamiento asíncrono	Carga de un documento de procesamiento prolongado, respuesta inmediata, cierre de la conexión y continuación del trabajo. Cuando se haya implementado, entrega duplicada que no genera un segundo bundle.
6. Evidencia de aislamiento		1,5 min	Aislamiento multiusuario	Intento de un usuario de acceder a un recurso ajeno y su denegación por parte del servidor.
7. Inspección y validación del bundle		2,5 min	Conversión documental, Validación del bundle	Apertura de index.md , log.md y los conceptos. Caso de documento breve con un concepto, caso con varias unidades ordenadas y caso de bundle incompleto que no se publica.
8. Recorrido por el código en Go		3 min	Backend en Go, Procesamiento asíncrono	Endpoint de carga que encola y retorna, publicación en la cola, consumo por el worker, idempotencia y manejo de errores.
9. Cierre		1,5 min	Encuadre general	Limitaciones conocidas, trabajo pendiente y reflexión sobre el uso de agentes de inteligencia artificial.

10.3 Cobertura de las condiciones verificables

Las seis condiciones de la sección 6 deben aparecer en el video. La siguiente tabla indica su ubicación esperada.

La calificación se efectúa recorriendo la tabla de la sección 10.2. Cada segmento aporta la evidencia del criterio asociado y el evaluador asigna el nivel correspondiente en la rúbrica.

11. Recomendaciones

- Se recomienda iniciar por el flujo asíncrono de extremo a extremo con un conversor trivial, aunque se limite a copiar el texto en un único concepto. Disponer del bundle mínimo y del despliegue en funcionamiento desde la primera semana reduce el riesgo del proyecto.

- Conviene tratar los documentos de entrada como no confiables y validar su formato y tamaño en la recepción.
- Se recomienda conservar en el trabajo y en el almacenamiento toda la información que un worker requiere, y no en la memoria de la API.
- Conviene verificar la idempotencia y el aislamiento antes de la sustentación, dado que son las condiciones que con mayor frecuencia se omiten.